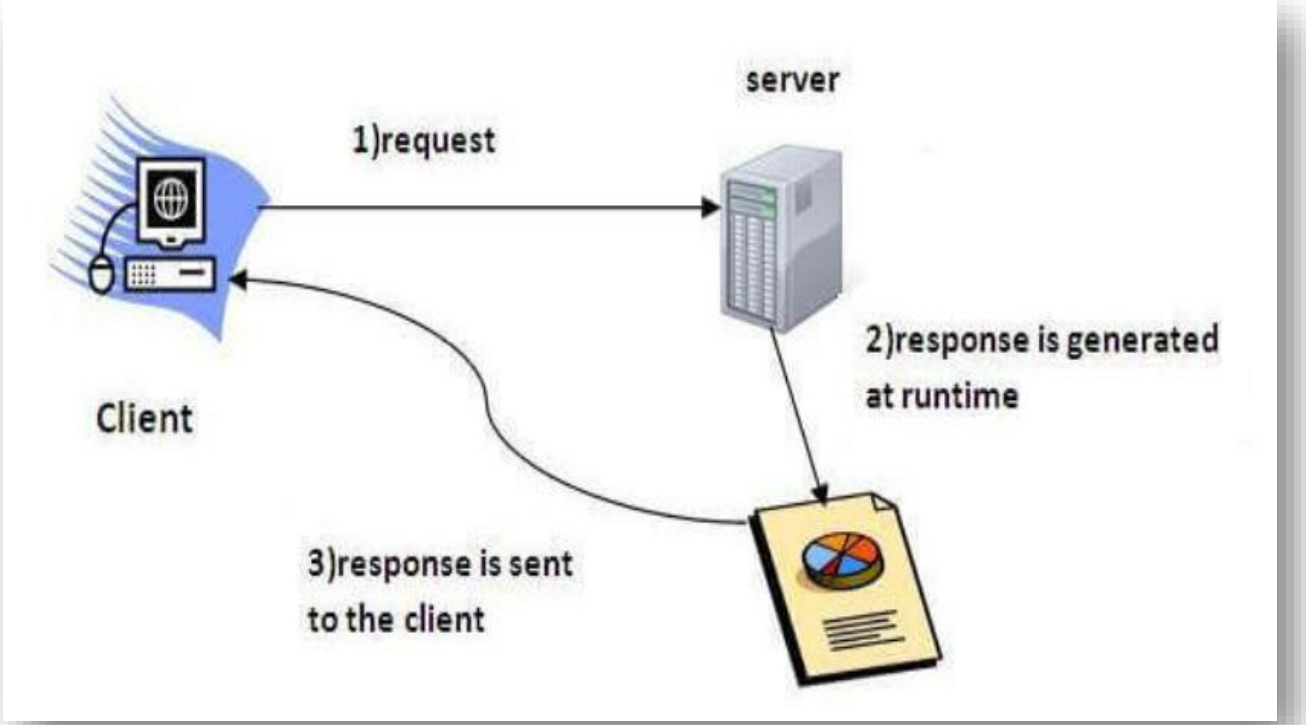
SERVLETS

- **Servlet is a technology used to create a web application.**
- **Servlet is a class that runs on web server.**
- **Web Application : Application which runs on server and need internet is known as web application.**
- **Server : It is a computer which provides service to other computers. Server accepts then request sent by the client , processes it , generates the response , and send back response to the client.**



Servlet API

- The javax.servlet and javax.servlet.http packages represent interfaces and classes for servlet api.

- The javax.servlet package contains many interfaces and classes (like , Servlet interface , GenericServet class etc) that are used by the servlet or web container. These are not specific to any protocol.

- The javax.servlet.http package contains interfaces and classes(like, HttpServlet class etc) that are responsible for http requests only.

WAYS TO CREATE A SERVLET

- **Create a class :**

1. **by implementing Servlet interface**
2. **by extending GenericServlet class.**
3. **by extending HttpServlet class.**

SERVLET INTERFACE

- **Servlet interface provides common behavior to all the servlets**

.

- **Servlet interface defines methods that all servlets must implement.**

- **Servlet interface needs to be implemented for creating any servlet (either directly or indirectly).**

- **It provides 3 life cycle methods that are used to initialize the servlet, to service the requests, and to destroy the servlet and 2 non-life cycle methods.**

Methods of Servlet interface

There are 5 methods in Servlet interface :

1. `public void init(ServletConfig config)` : **initializes the servlet. It is the life cycle method of servlet and invoked by the web container only once.**
2. `public void service(ServletRequest request,ServletResponse response)` : **accepts the request and provides response for the incoming request. It is invoked at each request by the web container.**
3. `public void destroy()` : **is invoked only once and indicates that servlet is being destroyed.**
4. `public ServletConfig getServletConfig()` : **returns the object of ServletConfig.**
5. `public String getServletInfo()` : **returns information about servlet such as writer, copyright, version etc.**

GenericServlet class

- **GenericServlet class implements Servlet interface. It provides the implementation of all the methods of these interfaces except the service method.**

- **GenericServlet class can handle any type of request so it is protocol-independent.**

- **We may create a generic servlet by inheriting the GenericServlet class and providing the implementation of the service method.**

Service() method to be overridden is :

- public void service(ServletRequest request,ServletResponse response) : **accepts the request and provides response for the incoming request. It is invoked at each request by the web container.**

- **The service() method accepts two parameters :**
- **ServletRequest (interface) : An object of ServletRequest is used to provide the client request information to a servlet such as parameter names and values, attributes etc.**
  - **It consists of a method : public String getParameter(String name) : it is used to obtain the value of a parameter by name.**
  - **Etc...**

- **ServletResponse (interface) : An object of ServletRequest is used to format and send the response back to the client.**
  - **It consists of methods : void setContentType(String) : Sets the type of the response data. e.g. text, html etc.**
  - **java.io.PrintWriter getWriter() : Returns the PrintWriter object.**
  - **Etc..**

HttpServlet class

- **The HttpServlet class extends the GenericServlet class.**
- **It has all the properties inherited from GenericServlet class.**
- **In HttpServet class , service() method is also overridden.**
- **Along with the inherited methods , It provides http specific methods such as doGet, doPost, doHead, doTrace etc to handle specific type of requests.**

THE METHODS TO HANDLE DIFFERENT REQUESTS ARE :

- protected void doGet(HttpServletRequest req, HttpServletResponse res) **handles the GET request. It is invoked by the web container.**
- protected void doPost(HttpServletRequest req, HttpServletResponse res) **handles the POST request. It is invoked by the web container.**
- protected void doPut(HttpServletRequest req, HttpServletResponse res) **handles the PUT request. It is invoked by the web container.**
- protected void doDelete(HttpServletRequest req, HttpServletResponse res) **handles the DELETE request. It is invoked by the web container.**
- **Etc...**

These methods accepts two parameters :

- **HttpServletRequest (interface) : Extends the ServletRequest interface to provide request information for HTTP servlets.**
- **An object of HttpServletRequest is used to provide the client request information to a Http servlet such as parameter names and values, attributes etc.**

  It inherits the methods of ServletRequest interface such as :

  - **public String getParameter(String name)**
  - **Etc...**

- **HttpServletResponse (interface) : Extends the ServletResponse interface to provide HTTP-specific functionality in sending a response.**
- **An object of HttpServletResponse is used to format and send the response back to the client from HTTP servlets.**

  It inherits the methods of ServletResponse interface such as :

  - **void setContentType(String) : Sets the type of the response data. e.g. text, html etc.**
  - **java.io.PrintWriter getWriter() : Returns the PrintWriter object.**
  - **Etc..**

sendRedirect() method

- **It is a method defined in HttpServletResponse interface in java.**
- **It is used to redirect the client to a different URL.**

Syntax : void sendRedirect(String location) throws IOException

- **Parameters:**

  location: The URL to which the client is redirected. It can be an absolute or relative URL.

It can be used for navigating within the same server as well as outside the server.

NOTE :

When to Use:

1. **forward() : When transferring control to another resource within the same application and When you want to share request data with the target resource.**

   → Supported by GenericServlet class and HttpServlet class.

2. **sendRedirect() : When redirecting to an external resource or a different application running on different server.**

   → Supported only in HttpServlet class.

WEB CONTAINER

- It is a component of web server that manages the servlets and their entire life cycle.

- Web container creates servlet instances, loads and unloads servlets, creates and manages request and response objects, and performs other servlet-management tasks.

- It is also responsible to map the URL with the corresponding servlet.

@WebServlet Annotation

- @WebServlet annotation is used to declare a servlet.

- This annotation is specified on a class and contains metadata about the servlet being declared.

- The annotated servlet must specify at least one URL pattern.

PRINTWRITER

- The Java PrintWriter class implements the Writer class. It is used to print the formatted representation of objects to the text output stream.

- It provides convenient methods for writing various forms of data such as integer, double, and String or Object in the form of text the browser , in to a file or any other output destination.

- It consists of print() method and write() methods for printing / displaying data .

- In servlets , we get the PrintWriter object using getWriter() of ServlerResponse interface.

  Ex : PrintWriter ref = response.getWriter();

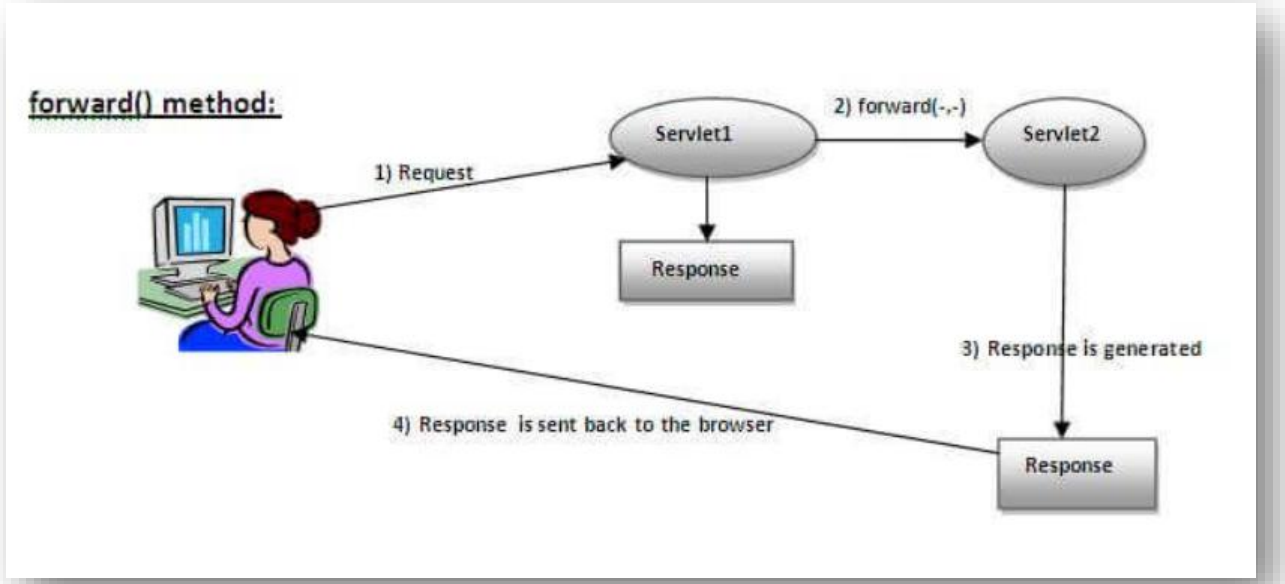
REQUEST DISPATCHER

- **The RequestDispatcher interface provides the facility of dispatching the request(forwarding) to another resource it may be html, servlet or jsp.**
- **This interface can also be used to include the content of another resource also.**

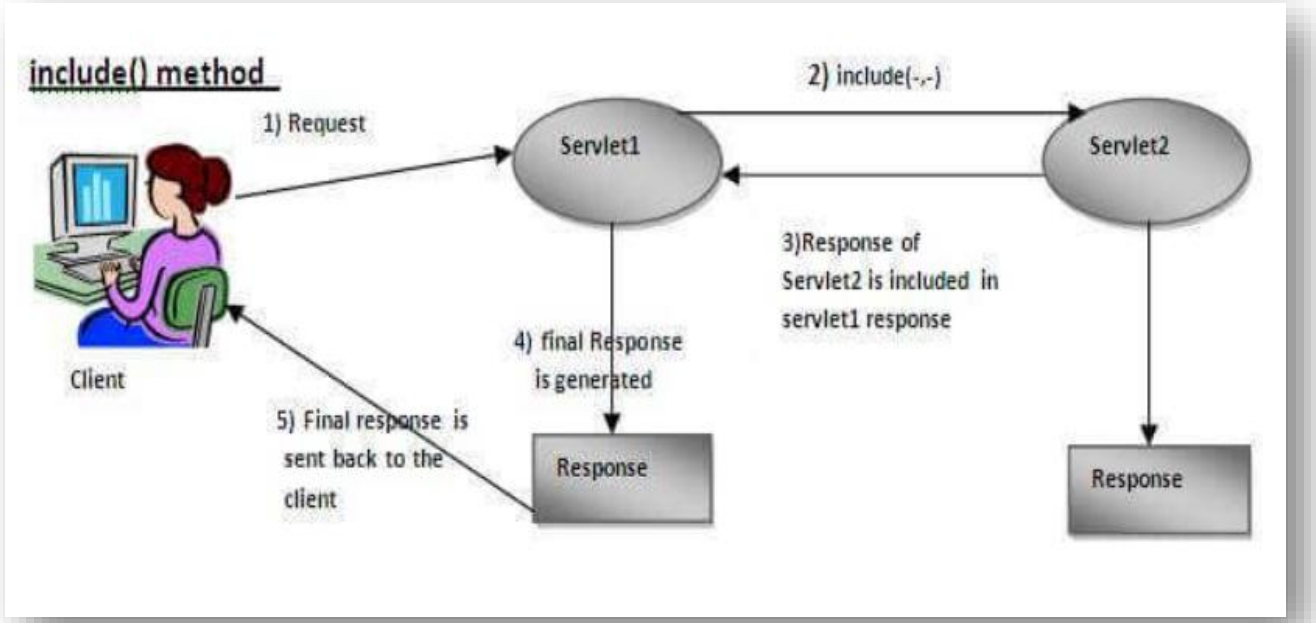
Methods of RequestDispatcher interface

- **public void forward(ServletRequest request,ServletResponse response) : Forwards a request from a servlet to another resource (servlet, JSP file, or HTML file) on the server.**

- **public void include(ServletRequest request,ServletResponse response) :Includes the content of a resource (servlet, JSP page, or HTML file) in the response.**



Response of second servlet is sent to the client. Response of the first servlet is not displayed to the user. It is forwarded to the second servlet.



Response of second servlet is included in the response of the first servlet that is being sent to the client.

How to get the object of RequestDispatcher

The getRequestDispatcher() method of ServletRequest interface returns the object of RequestDispatcher.

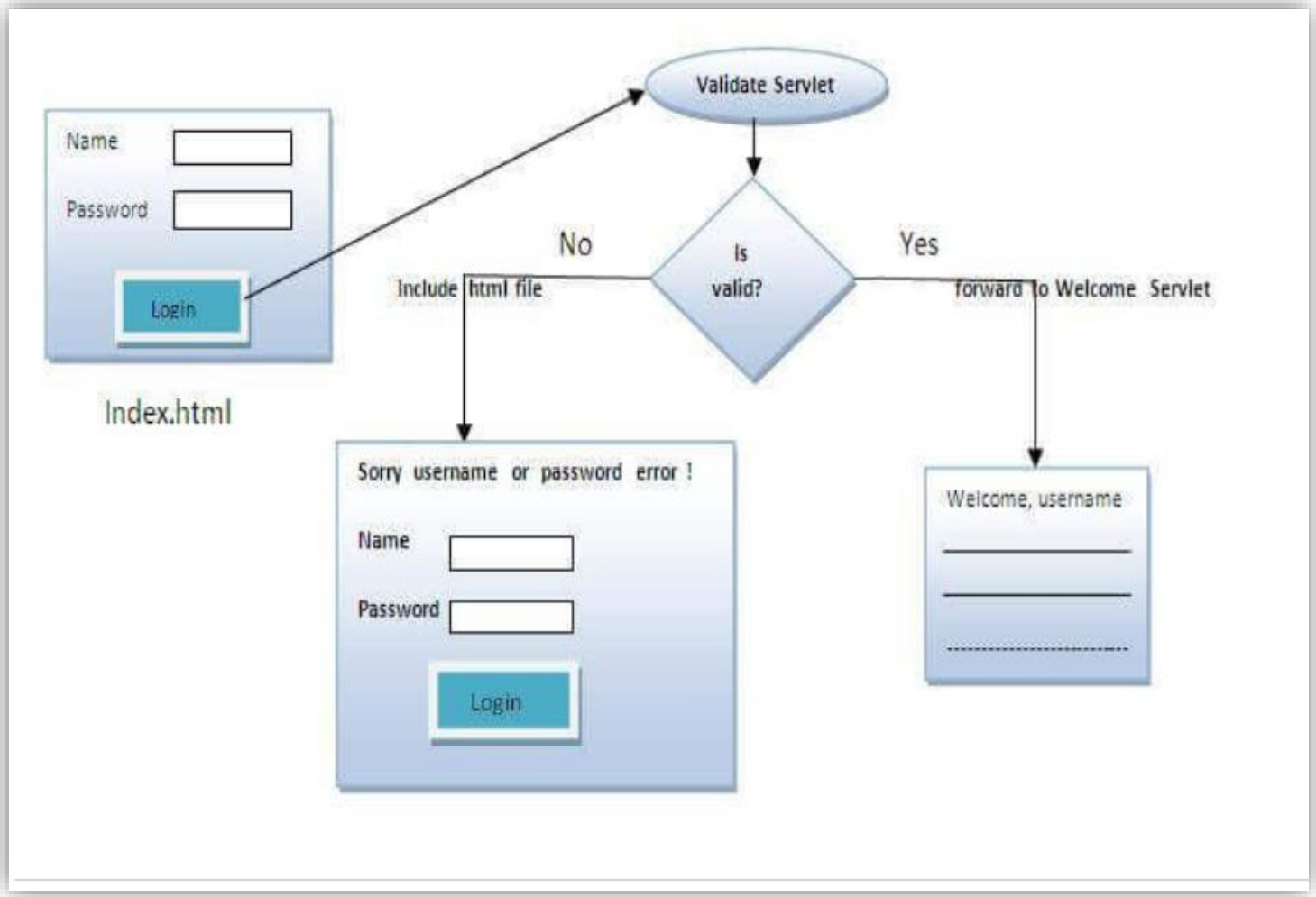
Syntax :

```
public RequestDispatcher getRequestDispatcher(String resource);
```

Example :

```
RequestDispatcher rd=request.getRequestDispatcher(resource);
//resource can be url-pattern of the servlet  , jsp or html file name.
```

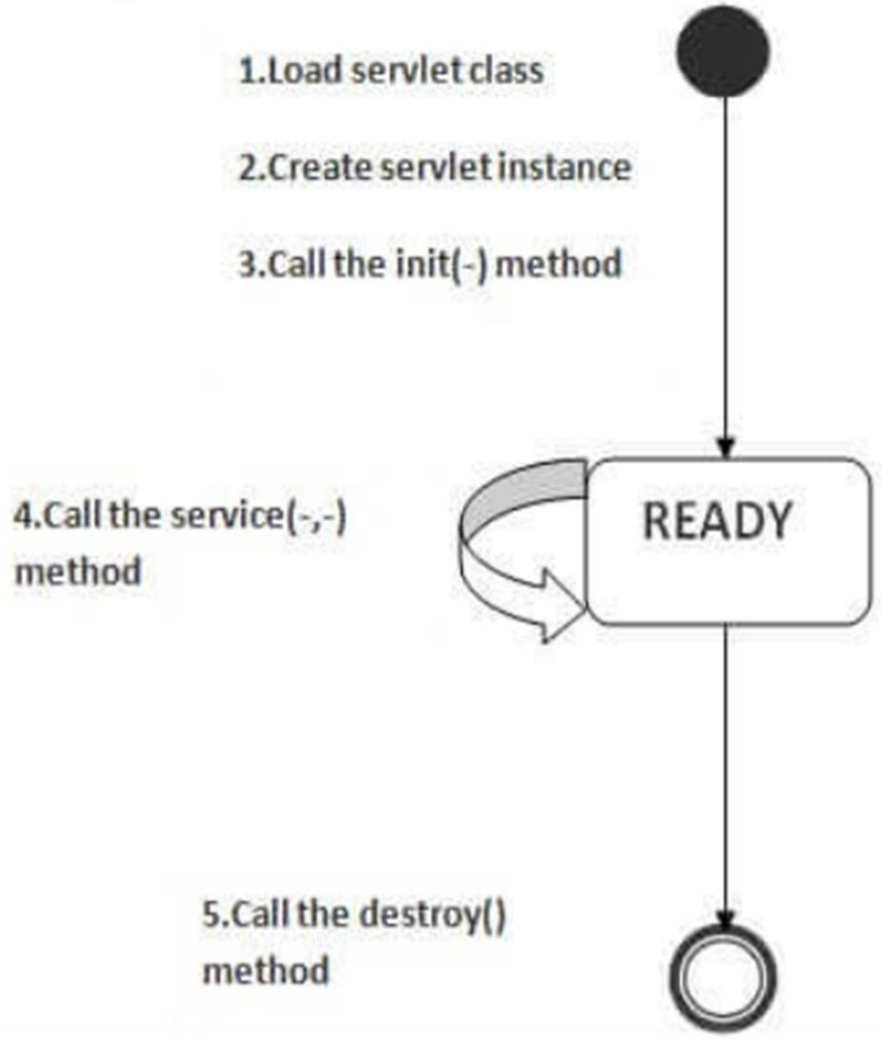
Example :



In this example, we are validating the password entered by the user. If password is correct as per the condition, it will forward the same request to the WelcomeServlet, otherwise it will include an error message : " sorry username or password error! " in login html file.

Life Cycle of a Servlet (Servlet Life Cycle)

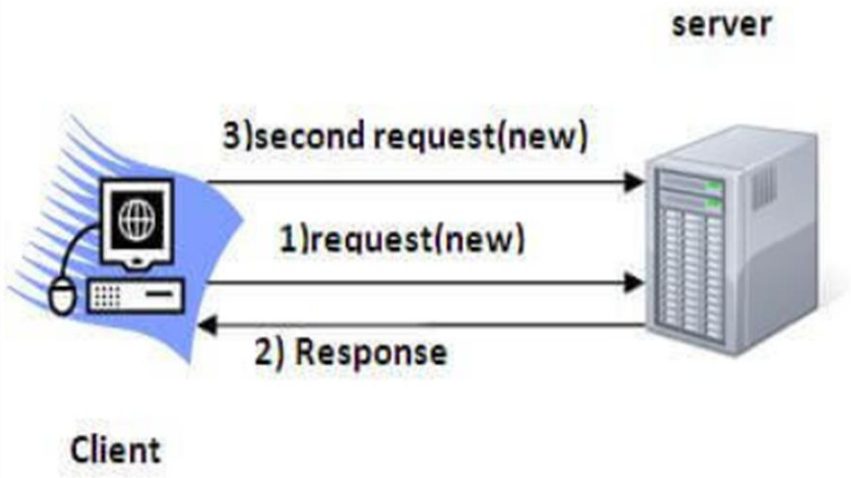
- **Servlet class is loaded.**
- **Servlet instance is created.**
- **init method is invoked.**
- **service method is invoked.**
- **destroy method is invoked.**



- **1) Servlet class is loaded :** The servlet class is loaded when the first request for the servlet is received by the web container.
- **2) Servlet instance is created :** The web container creates the instance of a servlet after loading the servlet class. The servlet instance is created only once in the servlet life cycle.
- **3) init method is invoked :** The web container calls the init method only once after creating the servlet instance. The init method is used to initialize the servlet.
- **4) service method is invoked :** The web container calls the service method each time when request for the servlet is received.
- **5) destroy method is invoked :** The web container calls the destroy method before removing the servlet instance from the service. It gives the servlet an opportunity to clean up any resource for example memory, thread etc.

Session Tracking

- Session simply means a particular interval of time.
- Session Tracking is a way to maintain state (data) of an user. It is also known as session management in servlet.
- Each time user requests to the server, server treats the request as the new request, i.e.. Server doesn't recognize the user.
- So we need session tracking to maintain the state of an user for the server to recognize the particular user.



Session Tracking Techniques

- There are four techniques used in Session tracking:

1. Cookies
2. Hidden Form Field
3. URL Rewriting
4. HttpSession

Cookies in Servlet

- **A cookie is a small piece of information that is persisted(stored) between the multiple client requests.**

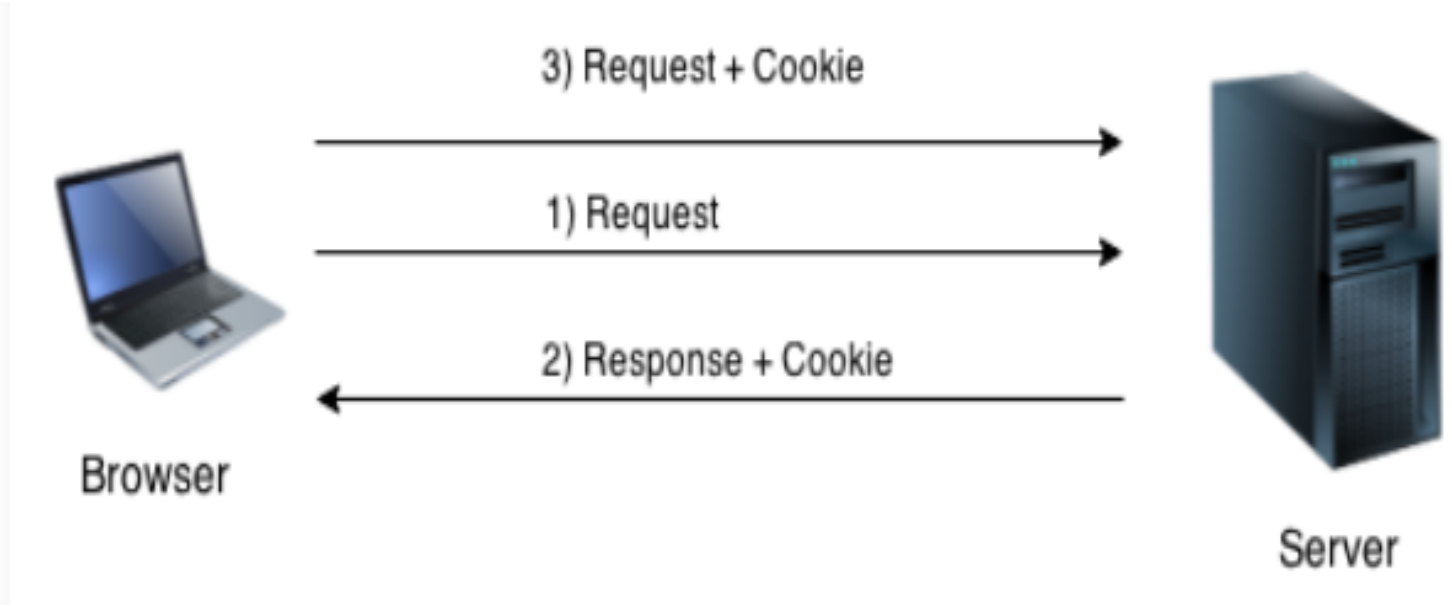
- **A cookie has a name, a single value, and optional attributes such as a comment, path , a maximum age etc.**

How Cookie works

- **By default, each request is considered as a new request. In cookies technique, when client sends the first request to the server, a cookie is created in the server in which client information is stored.**

- **When server sends back response to the client , we add cookie with the response from the server.**

- **So cookie is stored in the cache memory of the browser. After that if request is sent by the user, cookie is added with request by default. Thus, the server is able to recognize the user as the old user.**



Types of Cookie

- **There are 2 types of cookies in servlets :**

1. **Non-persistent cookie**
2. **Persistent cookie**

• Non-persistent cookie

It is valid for single session only. It is removed each time when user closes the browser.

• Persistent cookie

It is valid for multiple session . It is not removed each time when user closes the browser. It is removed only if user logout or signout.

Advantage of Cookies

- **Simplest technique of maintaining the state.**
- **Cookies are maintained at client side.**

Disadvantage of Cookies

- **They're browser dependent. Hence, if the client disables them, your web application can't make use of them.**
- **To overcome this , we go for HttpSession technique.**

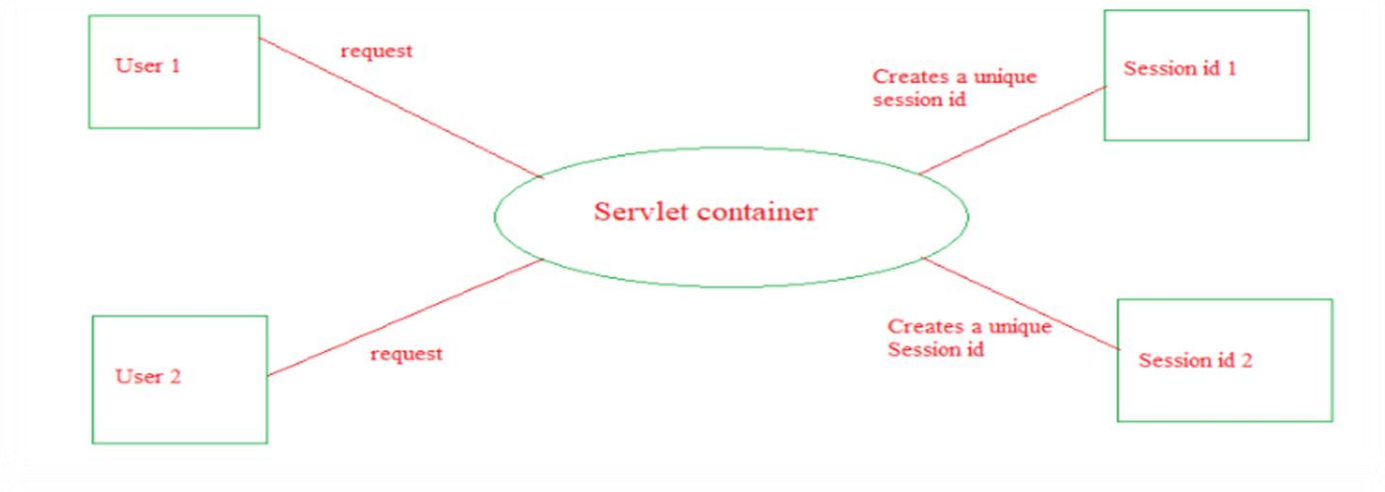
HttpSession in Servlet

- **A session is simply the limited interval of time in which two systems communicate with each other..**

- **HttpSession is an interface provided by the javax.servlet.http package that allows you to store and retrieve user-specific data across multiple requests made by the same client (typically a browser).**

How It works

- **When a client connects to a web application:**
- **A session is created by the server using request.getSession().**
- **A unique session ID is generated and sent to the client (SESSIONID).**
- **On subsequent requests, the session ID is sent back to the server to retrieve the session.**



COMMON METHODS

```
HttpSession session = request.getSession(); // creates new session if not already created
```

- `// Store data`

```
session.setAttribute("username", "pooja");
```

- `// Retrieve data`

```
String user = (String) session.getAttribute("username");
```

- `// Invalidate the entire session`

```
session.invalidate();
```

- `// Get session ID`

```
session.getId();
```